



Hochschule Macromedia für angewandte Wissenschaften
University of Applied Sciences

Kursbezeichnung: Game Programmierung
Prüfer: Jonas Heinke

Vom Studierenden auszufüllen:

306596	L-UBf GD GDS 6d 24W
Matrikelnummer	Kohorte
Kletzander	Darian
Nachname	Vorname

Die Arbeit wird eingereicht als:
(Tragen Sie bitte in die zutreffende Box den Buchstaben X ein)

☒ Einzelarbeit
☐ Gruppenarbeit

Trifft nur auf Gruppenarbeiten zu: (Nur bei Gruppenarbeiten auszufüllen)

Falls Sie eine Gruppenarbeit einreichen, dann müssen bitte die Vor- und Nachnamen aller Gruppenmitglieder aufgeführt werden. Die Namen sind von den jeweiligen Gruppenmitgliedern selbst elektronisch einzutragen. Durch Eintrag des Namens wird bestätigt, dass der/die Studierende mit der Abgabe der Arbeit in der vorliegenden Form einverstanden ist. Ferner wird mit Eintrag des Namens erklärt, die Projektarbeit (bei Gruppenarbeit: den von dem/der jeweiligen Studierenden erstellten und entsprechend gekennzeichneten Teil der Arbeit selbständig und ohne fremde Hilfe angefertigt zu haben. Dabei hat der/die Studierende sich keiner anderen Hilfsmittel bedient als derjenigen, die im beigelegten Quellenverzeichnis genannt sind. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind von dem/der Studierenden als solche kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Mit der Abgabe stimmen die Gruppenmitglieder zu, dass alle Bewertungen und Hinweise der Prüfer:innen in der hochgeladenen Arbeit hinterlegt werden. Das Gruppenmitglied, das den Upload vorgenommen hat, muss die Korrekturhinweise, den anderen Gruppenmitgliedern zugänglich machen.

1) _____
2) _____
3) _____
4) _____

5) _____
6) _____
7) _____
8) _____

Bewertung der Gruppenarbeiten:

(Tragen Sie bitte in die zutreffende Box den Buchstaben X ein)

☐ Ich beantrage bei meiner Gruppenarbeit eine Individualbewertung
☐ Ich beantrage bei meiner Gruppenarbeit eine Gruppenbewertung

Eine Individualbewertung bedeutet, dass jedes Gruppenmitglied eine individuelle Note bekommt und eine Gruppenbewertung bedeutet, dass jedes Gruppenmitglied eine identische Note bekommt.

Teuchern, 15.07.2026

Darian Kletzander

Ort/Datum

Vollständiger Vor- und Nachname

Bewertung (gemäß Notenschlüssel), Ergebnis Erstprüfung: Gesamtpunkte: _____

Datum:

Name, Vorname Erstprüfender (elektronisch auszufüllen)

Vom Prüfer auszufüllen: (Freitext für die Zweitkorrektur)

Technical Design Document

Code und Gestaltung der Abgaben A1 bis A_Final

CODE / ARCHITEKTUR / GAME DESIGN

Die Abgaben zeigen die Entwicklung von grundlegenden Python-Strukturen bis zu einem vollständig modularisierten Arcade-Spiel mit pygame-ce. Im Mittelpunkt stehen Programmaufbau, Zuständigkeiten der Module, Datenfluss, Spielmechaniken und visuelle Gestaltung.

Projektüberblick

Abgabe	Technischer Schwerpunkt	Gestalterischer Schwerpunkt
A1	Schleifen, Bedingungen, Funktionen und Rückgabewerte	Klare Konsolenausgabe und nachvollziehbarer Programmablauf
A2	Game Loop, Bewegung, Gravitation und Rechteckkollision	Jump-and-Run-Szene mit Plattformen, Figur, Sound und Sammelobjekten
A3	Klassen, modulare Spiellogik und datenbasierte Level	Gegner, Hindernisse, HUD und Game-Over-Zustand
A4	Score, Shop, Levelprogression und Boss-System	Scrollende Hintergründe, steigende Schwierigkeit und Bosskämpfe
A_Final	State Machine, getrennte Systeme und persistente Daten	Prozedurales Science-Fiction-Arcade-Spiel mit drei Sektoren

A1 - Schleifen und Funktionen

Ziel und Aufbau

A1 besteht aus zwei voneinander getrennten Rechenaufgaben. Der erste Teil summiert einen festgelegten Ganzzahlenbereich. Der zweite Teil kapselt einen Vergleich zweier Fließkommazahlen in einer Funktion und liefert das Verhältnis des kleineren zum größeren Wert zurück.

- **Summenbildung:** summe startet bei 0. Die for-Schleife durchläuft `range(437, 32483)`, weil die obere Grenze von `range` nicht eingeschlossen wird.
- **Funktionsschnittstelle:** `teile_kleineren_durch_groesseren(wert_1: float, wert_2: float) -> float` beschreibt Parameter und Rückgabewert direkt über Typangaben.
- **Fallunterscheidung:** `if` und `elif` bestimmen den größeren Wert. Bei Gleichheit liefert der `else`-Zweig 1.0.
- **Datenfluss:** Eingabewerte werden nicht verändert; die Funktion berechnet ausschließlich einen Rückgabewert.

Algorithmische Struktur

Schritt	Codeprinzip	Bedeutung
1	Initialisierung	Die Akkumulatorvariable erhält einen definierten Startwert.
2	Iteration	Jede Ganzzahl des Bereichs wird genau einmal verarbeitet.
3	Akkumulation	Der aktuelle Wert wird zur bisherigen Summe addiert.
4	Verzweigung	Die Funktion wählt abhängig von der Größenrelation einen Rechenweg.
5	Rückgabe	Das berechnete Verhältnis wird als float zurückgegeben.

A2 - Jump & Run mit pygame-ce

Spielidee und Gestaltung

A2 ist ein kompaktes Jump-and-Run in einem festen Fenster. Der Spieler bewegt eine Pixel-Art-Figur über Plattformen, springt und sammelt fallende rote Kugeln innerhalb eines Zeitlimits. Ein beweglicher Kreis dient als zusätzliches Kollisionsobjekt. Klare Farben und einfache Formen halten die Spielsituation jederzeit lesbar.

Code-Struktur

- **Konstanten:** Bildschirmgröße, Farben, Gravitation, Sprunggeschwindigkeit und Zeitwerte stehen gesammelt am Dateianfang.
- **Zustandsdaten:** Positionen, Geschwindigkeiten, Punktestand, Timer und Statuswerte werden in einem gemeinsamen Laufzeitzustand geführt.
- **Kollision:** `player_collides_with()` kapselt `pygame.Rect.collidect()`. `move_and_collide()` verarbeitet Bewegung getrennt auf X- und Y-Achse.
- **Objekterzeugung:** `spawn_collectible()` erzeugt Sammelobjekte oberhalb des sichtbaren Bereichs und übergibt sie an die laufende Objektliste.
- **Persistenz:** Der Highscore wird in `highscore.json` geladen und bei einem neuen Bestwert aktualisiert.

Game Loop

Phase	Aufgabe
Event	Tastatur, Fensterereignisse und Sprunganforderung erfassen.
Update	Gravitation, Bewegung, Kollisionen, Sammelobjekte und Zeitlimit aktualisieren.
Draw	Plattformen, Spielfigur, Kreis, Sammelobjekte und Anzeigen zeichnen.
Frame-Begrenzung	<code>pygame.time.Clock</code> stabilisiert die Bildrate und den zeitlichen Ablauf.

Steuerung und Medien

Eingabe / Datei	Funktion
A / D	Horizontale Bewegung
W / Leertaste	Sprung, wenn die Figur auf einer Fläche steht
Escape	Spiel beenden
<code>assets/player.png</code> / <code>jump.wav</code>	Pixel-Art-Spielfigur und akustisches Feedback beim Sprung

A3 - RealFakeGame Teil 1

Spielidee und Gestaltung

A3 erweitert das Grundgerüst zu einem vertikalen Actionspiel. Der Player folgt der Mausposition auf der X-Achse und feuert automatisch. Gegner erscheinen zeitgesteuert aus den Leveldaten. Hindernisse wirken als Upgrades oder Downgrades. HUD, Trefferfeedback und Game Over bilden einen vollständigen Spielzustand.

Modulare Architektur

Modul	Verantwortung
entity.py	Gemeinsame Basisdaten und Verhalten beweglicher Spielobjekte
player.py	Position, Lebenspunkte, Waffe und Spielerzustand
shot.py	Automatisch erzeugte Projektile und deren Bewegung
enemy.py	Gegnerbewegung, Schaden, Lebenspunkte und Zielverfolgung
obstacle.py	Rechteckgeometrie, Scrollen, Kollision und Statuseffekt
level.py	Einlesen der .rfg-Datei und zeitgesteuertes Aktivieren von Objekten
main.py	Game Loop, Kollisionen, HUD, Neustart und Zustandswechsel

Datenfluss und Spielregeln

- **Leveldaten:** E-Zeilen beschreiben Gegner mit Spawnzeit, Position, Schaden und Geschwindigkeit. Hindernisse werden ebenfalls aus der Leveldatei aufgebaut.
- **Aktivierung:** Level.step() schaltet Objekte anhand ihrer Zeitposition frei und aktualisiert aktive Gegner relativ zum Player.
- **Trefferauflösung:** Die zentrale Kollisionsfunktion ordnet Player, Shots, Enemies und Obstacles einander zu und entfernt verbrauchte Objekte.
- **Statusanzeige:** Lebenspunkte, Waffenwerte und kurze Meldungen geben direkte Rückmeldung zu Schaden und eingesammelten Effekten.
- **Zustandswechsel:** Bei null Lebenspunkten wechselt das Spiel in Game Over; ein Neustart setzt Player, Level und Objektlisten zurück.

A4 - RealFakeGame Teil 2

Erweiterte Spielschleife

A4 ergänzt A3 um drei aufeinanderfolgende Level, Punkte, Combo, Highscore, In-Game-Währung, einen Upgrade-Shop und Bosskämpfe. Die Schwierigkeit steigt durch dichtere Gegnerwellen, höhere Geschwindigkeiten, ZigZag-Gegner und widerstandsfähigere Bosse.

Neue Kernmodule

Modul	Verantwortung
score.py	Punktestand, Combo-Fenster, Währung und persistenter Highscore
shop.py	Upgrade-Angebote, Kosten, Levelgrenzen und Anwendung auf Player-Statistiken
boss.py	BossEnemy, BossProjectile, Phasenverhalten und Mehrfachsalven
level.py	Mehrere Leveldateien, Hintergrund-Tiles, Spezialgegner und Boss-Einträge
main.py	Zustände playing, shop, gameover und won sowie deren Übergänge

Progression, Shop und Bosssdesign

- **Levelprogression:** Drei .rfg-Dateien definieren unterschiedliche Wellen, Geschwindigkeiten und Bosswerte. Der nächste Abschnitt beginnt erst nach Abschluss des aktuellen Levels.
- **Scrollender Hintergrund:** Zwei kleine Tile-Grafiken bilden eine wiederholbare Folge und werden abhängig von der Levelposition über den sichtbaren Bereich verteilt.
- **Wirtschaftssystem:** Besiegte Gegner geben Score und Currency. Der Shop berechnet Kosten aus Upgrade-Typ und aktueller Stufe.
- **Upgrade-Wirkung:** Kaufbare Werte sind Schaden, Feuerrate, Lebenspunkte und Projektilgeschwindigkeit. HP-Upgrades erhöhen max_hp und heilen den Player.
- **Bosskampf:** Der Boss bleibt im oberen Spielfeld, bewegt sich horizontal, feuert drei ausgerichtete Projektile und besitzt eine eigene Lebensanzeige.
- **Lesbarkeit:** Kontrastreiche Shopkarten, Kosten, aktuelle Werte und Boss-HP trennen Meta-Progression klar vom laufenden Kampf.

Steuerung

Die Mausposition steuert den Player horizontal. Im Shop wählen die Tasten 1 bis 4 ein Upgrade. Enter oder Leertaste startet das nächste Level, R beginnt nach Game Over oder Sieg neu und Escape beendet das Spiel.

A_Final - Signal Breach

Spielkonzept und visuelle Richtung

Signal Breach ist ein vertikaler 2D-Arcade-Shooter ohne Game Engine. Der Player steuert ein Reparaturprogramm durch drei beschädigte Datensektoren. Jeder Sektor kombiniert zeitgesteuerte Gegnerwellen mit einer eigenen Sektorwache. Dunkle Blautöne, Cyan-Akzente, geometrische Formen und dezente Partikel erzeugen eine klare Science-Fiction-Oberfläche.

- **Kernschleife:** Kämpfen, Datenfragmente sammeln, einen Sektorboss besiegen, eine Upgrade-Karte wählen und den nächsten Sektor starten.
- **Kampfgefühl:** Automatisches Feuer hält den Fokus auf Bewegung und Ausweichen. Der Impuls entfernt nahe feindliche Projektile und besitzt eine Abklingzeit.
- **Progression:** Drei Sektoren erhöhen Gegnerdichte, kombinieren zusätzliche Gegnertypen und verändern die Bossmuster.
- **Darstellung:** Schiff, Gegner, Projektile, HUD, Shopkarten und Effekte werden direkt mit pygame-Zeichenfunktionen erzeugt.
- **Audio:** Kurze Signale entstehen mathematisch als PCM-Samples; das Spiel benötigt keine externen Sounddateien.

Systemarchitektur

Datei	Zuständigkeit
main.py	pygame-Initialisierung, skalierbares Fenster, virtueller Canvas und Hauptschleife
model.py	ScreenState, Profile, RunStats und PlayerStats als getypte Datenmodelle
game.py	State Machine, Eingabe-Actions, Simulation, Kollisionen und zentrale Übergänge
entities.py	Player, Projectile, Enemy und Boss ohne direkte Zeichenverantwortung
levels.py	Sektordaten, Wellen, Formationen und zeitgesteuerte Freigabe des Bosses
scoring.py	Score, Combo, Datenfragmente, Upgrade-Draft und Statuseffekte
events.py	Kleiner EventBus für entkoppelte Reaktionen zwischen Systemen
rendering.py	Menü, HUD, Spielfeld, Zustandsanzeigen, Gegner und Shopkarten
audio.py	Prozedurale Sounderzeugung und stummer Fallback ohne Audiogerät
persistence.py	Validiertes Laden und atomisches Speichern des Profils als JSON

Zustandsmodell

ScreenState und change_state() bilden die erlaubten Übergänge explizit ab. Dadurch bleibt eindeutig, welche Eingaben und Aktualisierungen in einem Zustand aktiv sind.

Zustand	Aufgabe	Typische Übergänge
MENU	Titel, Steuerung und gespeicherte Bestwerte anzeigen	PLAYING
PLAYING	Simulation, Kollisionen, Score und Wellen aktualisieren	PAUSED, SHOP, GAME_OVER, WON
PAUSED	Simulation anhalten und Overlay anzeigen	PLAYING, MENU
SHOP	Drei Upgrade-Karten anbieten und einen Kauf erlauben	PLAYING, WON
GAME_OVER	Run abschließen und Rückkehr anbieten	MENU
WON	Abschlusswerte und Rückkehr anbieten	MENU

Eingabe und Steuerung

Eingabe	Funktion
WASD / Pfeiltasten	Schiff bewegen
Mausbewegung	Schiff alternativ direkt positionieren
Leertaste	Impuls gegen nahe feindliche Projektile
P / Escape	Pause umschalten
1 bis 3	Eine der drei Upgrade-Karten im Shop wählen
Enter	Starten, bestätigen oder nächsten Sektor laden
F	Vollbild umschalten
M	Ton ein- oder ausschalten
R	Nach Sieg oder Niederlage zum Menü zurückkehren

Entities und Kampfsystem

- **Player:** PlayerStats bündelt Lebenspunkte, Schaden, Feuerrate, Projektiltempo, Reichweite, Streuschüsse und Impulsradius. Upgrades verändern diese Werte datenbasiert.
- **Projektile:** Player- und Gegnerprojektile besitzen Position, Geschwindigkeit, Schaden, Reichweite und Zugehörigkeit. Die maximale Flugstrecke begrenzt ihre Lebensdauer.
- **Gegnertypen:** Drifter, Weaver, Charger, Splitter und Fragmente teilen eine Basisklasse, unterscheiden sich aber in Bewegung und Angriff.
- **Bosse:** Boss erweitert Enemy und wechselt abhängig von Lebenspunkten und Sektor zwischen gerichteten, radialen und unterstützenden Angriffen.
- **Kollisionen:** Die Simulation löst Treffer zentral auf, aktualisiert Lebenspunkte und erzeugt bei Bedarf Fragmente, Score-Ereignisse und Statusmeldungen.

Sektoren und Leveldaten

Die drei Sektoren sind als Datenstrukturen definiert. Eine Welle beschreibt Zeitpunkt, Gegnertyp, Anzahl, Formation und Werte. SectorRuntime verfolgt die vergangene Zeit, gibt jede Welle genau einmal frei und öffnet den Bosskampf erst, wenn alle normalen Gegner verarbeitet sind.

Sektor	Gegnerfokus	Bosscharakter
1	Drifter und Weaver	Einführung in gerichtete Projektilmuster
2	Charger und Splitter	Mehr Druck durch schnelle Angriffe und Fragmente
3	Kombination aller Gegnerarten	Dichtere und radiale Angriffsmuster

Score, Combo und Upgrades

- **ScoreSystem:** Schnelle Abschüsse erhöhen die Combo. Spieler-Treffer oder lange Pausen setzen sie zurück. Datenfragmente bilden die Shop-Währung.
- **Upgrade-Daten:** Jeder Eintrag enthält Bezeichnung, Beschreibung, Basiskosten, Maximalstufe, Seltenheit, Gewichtung und Statureffekt.
- **Draft:** Zwischen Sektoren werden drei unterschiedliche, noch verfügbare Karten gewichtet ausgewählt.
- **Kaufregel:** Die Tasten 1 bis 3 entsprechen den sichtbaren Karten. Pro Shopphase kann höchstens eine Karte gekauft werden; danach markiert die Oberfläche die Auswahl.
- **Weiterführung:** Enter startet nach der Auswahl oder ohne Kauf den nächsten Sektor.

Rendering und Oberfläche

- **Virtueller Canvas:** Alle Inhalte werden auf eine feste interne Auflösung gezeichnet und anschließend proportional in das skalierbare Fenster übertragen.
- **Trennung der Zuständigkeiten:** Entities aktualisieren ihre Simulation, während Renderer sämtliche sichtbaren Formen und Texte zeichnet.
- **HUD:** Lebenspunkte, Score, Combo, Datenfragmente, Sektorfortschritt und Boss-HP bleiben außerhalb des zentralen Kampfbereichs angeordnet.
- **Shopkarten:** Drei gleich große Karten zeigen Taste, Name, Seltenheit, Beschreibung, Stufe und Preis. Farbe signalisiert Kaufbarkeit und Auswahl.
- **Feedback:** Trefferblitze, Impulsring, Partikel, Statusmeldungen und Farbwechsel machen Zustandsänderungen unmittelbar sichtbar.

Persistenz und Portabilität

- **Profildaten:** Highscore, beste Combo und Audioeinstellung werden in signal_breach_profile.json gespeichert. Ungültige Werte werden beim Laden auf sichere Grenzen gesetzt.
- **Atomisches Speichern:** Neue Profildaten werden zunächst in eine temporäre Datei geschrieben und anschließend ersetzt. Dadurch bleibt die Hauptdatei bei einem Abbruch konsistent.
- **Projektpfade:** Laufzeitdateien werden relativ zum Projekt beziehungsweise zur gestarteten Datei adressiert; absolute Pfade eines bestimmten Computers sind nicht im Spielcode hinterlegt.
- **Abhängigkeit:** pygame-ce ist in pyproject.toml und uv.lock beschrieben. Grafik und Audio benötigen keine zusätzlichen Medienordner.

Code-Gestaltungsprinzipien

Prinzip	Anwendung
Klare Zuständigkeiten	Simulation, Darstellung, Audio, Leveldaten, Score und Persistenz liegen in getrennten Modulen.
Datenorientierung	Sektoren, Wellen und Upgrades werden über Datenstrukturen beschrieben statt über wiederholte Spezialfälle.
Explizite Zustände	ScreenState und erlaubte Übergänge verhindern widersprüchliche Menü- und Spielzustände.
Komposition	Game verbindet ScoreSystem, UpgradeSystem, AudioManager, EventBus und Renderer über schmale Schnittstellen.
Vererbung mit begrenztem Umfang	Boss nutzt Enemy als gemeinsame Basis; Unterschiede bleiben in spezialisiertem Verhalten.
Robuste Schnittstellen	Dataclasses und Typangaben machen Werte nachvollziehbar; Audio, Profildaten und Zustandswechsel besitzen kontrollierte Fallbacks.